D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY, KOLHAPUR

(An Autonomous Institute)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A MINI-PROJECT REPORT ON

PYTHON PROGRAMMING (APC COURSE)

SORTING ALGORITHM RACE

EMPIRICAL COMPARISON OF SIX SORTING METHODS ACROSS INPUT PATTERNS

Submitted by

Sr.	Roll No.	Student Name
1	151	Manjunath Rajkumar Gavandi
2	152	Balaji Ramraje Gaikwad

T.Y (CSE) Div: C

Under the guidance of

Mrs. S. V. Shinge

Department of Computer Science and Engineering

Academic Year: 2026-27

D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY, KOLHAPUR

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the mini-project work entitled "SORTING ALGORITHM RACE: EMPIRICAL COMPARISON OF SIX SORTING METHODS ACROSS INPUT PATTERNS" has been successfully carried out by the following students of the Department of Computer Science and Engineering, D. Y. Patil College of Engineering and Technology, Kolhapur, during the academic year 2026-27:

Sr.	Roll No.	Student Name
1	151	Manjunath Rajkumar Gavandi
2	152	Balaji Ramraje Gaikwad

The mini-project report is submitted in partial fulfilment of the requirements for the Python Programming (APC) course and has been reviewed for internal assessment.

Date: _____

Place: Kolhapur

Mrs. S. V. Shinge	**Dr. R. J. Dhanal**	**Dr. Sanjay S. Pawar**
Course Coordinator	Head of the Department	Principal
Department of CSE	Department of CSE	D. Y. Patil College of Engineering and Technology, Kolhapur

1. Introduction

Sorting Algorithm Race is a desktop application for observing how sorting methods behave as input size and input order change. The program benchmarks five hand-implemented algorithms alongside Python's built-in sorted() function, plots the measured times, and allows a user to sort or compare a custom integer array.

Benchmark datasets are generated reproducibly from seed 42. The five input patterns are random, already sorted, reversed, nearly sorted (about 1% of positions swapped), and many duplicates. For each pattern, the program tests sizes 100, 500, 1,000, 2,000, and 5,000. Each measurement is the median of five runs using time.perf_counter().

Every result is checked against Python sorted(input); the input list must remain unchanged, and each algorithm must return a separate result list. The report files and graph are produced from the same benchmark run.

Table 1.1: Technologies and Methods Used

Component	Technology / use
Programming language	Python 3
Graphical interface	Tkinter, ttk, and scrolledtext
Background processing	threading and queue for benchmark progress and GUI updates
Sorting methods	Bubble, insertion, bottom-up merge, iterative randomized quick sort, heap, and Python sorted()
Benchmark data and analysis	random, statistics, NumPy, and pandas
Charts	Matplotlib; embedded in the application and saved as PNG
Measurement	time.perf_counter(); median of five runs; fixed data seed 42
Output files	results.csv, benchmark_report.txt, user_array_report.txt, sorting_race.png

2. SYSTEM ARCHITECTURE DIAGRAM

The application uses a local desktop architecture. The Tkinter interface starts background work for benchmarking, while the algorithms, correctness checks, measurement, charting, and file output run within the same Python program.



Figure 2.1: Local Architecture and Benchmark Data Flow

The worker sends progress messages through a thread-safe queue; the Tkinter event loop reads them and updates the progress bar and log. When the run completes, the GUI displays the timing curves and the worker-generated report data is saved to disk.

2.1 Working of the System

1. Input preparation: Make Graph creates the five deterministic input patterns at each of the five configured sizes. A user may instead enter comma-separated integers.

2. Background execution: The GUI starts benchmark work on a worker thread so the window can continue to process events and show progress.

3. Sorting and measurement: Each eligible algorithm receives a fresh copy of the input. Five runs are timed with time.perf_counter(), and their median is recorded.

4. Validation: Each output is compared with sorted(input). The program also checks that the original input was not modified and that a new list was returned.

5. Presentation and saving: Results are shown in the progress/analysis area, plotted in five log-log panels, and saved as CSV, text reports, and a PNG chart. Custom-array comparisons are ranked and saved separately.

3. OBJECTIVES

- To implement and compare five standard sorting algorithms with Python's built-in sorted() baseline.
- To measure sorting performance across random, sorted, reversed, nearly sorted, and duplicate-heavy input patterns.
- To test multiple input sizes using reproducible generated datasets and repeated timing measurements.
- To verify every output against a trusted sorted result and ensure that source input lists are preserved.
- To provide a graphical desktop interface for starting benchmarks, following progress, viewing charts, and sorting a user-provided array.
- To present results as a log-log comparison chart and export detailed measurements and readable text reports.

4. IMPLEMENTATION: OUTPUT SCREENSHOTS

The Python application was run locally as a desktop GUI. Its main window combines benchmark controls, the custom-array form, the progress and analysis log, and the timing chart. The screenshot shows the sample input [8, 3, 5, 1, 9] and the generated performance graph.

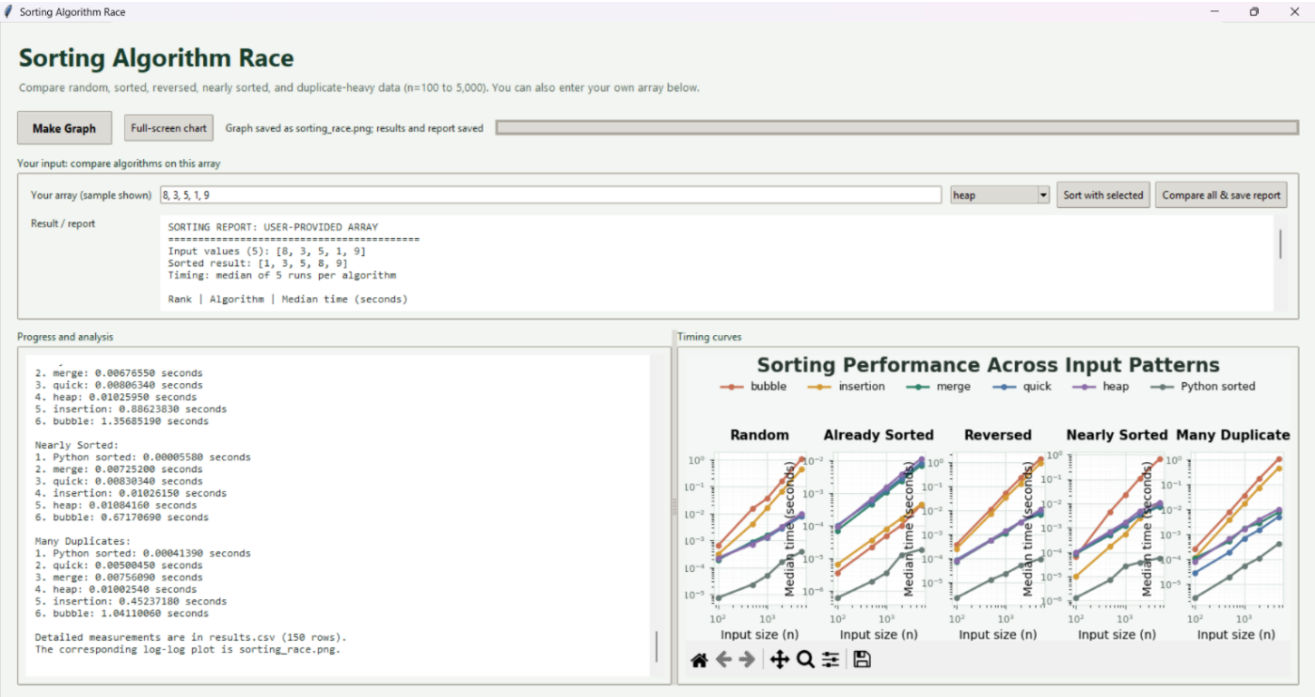


Figure 4.1: Sorting Algorithm Race Main Window and Sample Output

The benchmark chart compares median execution time against input size for each of the five input patterns. Both axes use logarithmic scales so that the growth of fast and slow methods can be compared on the same panel.

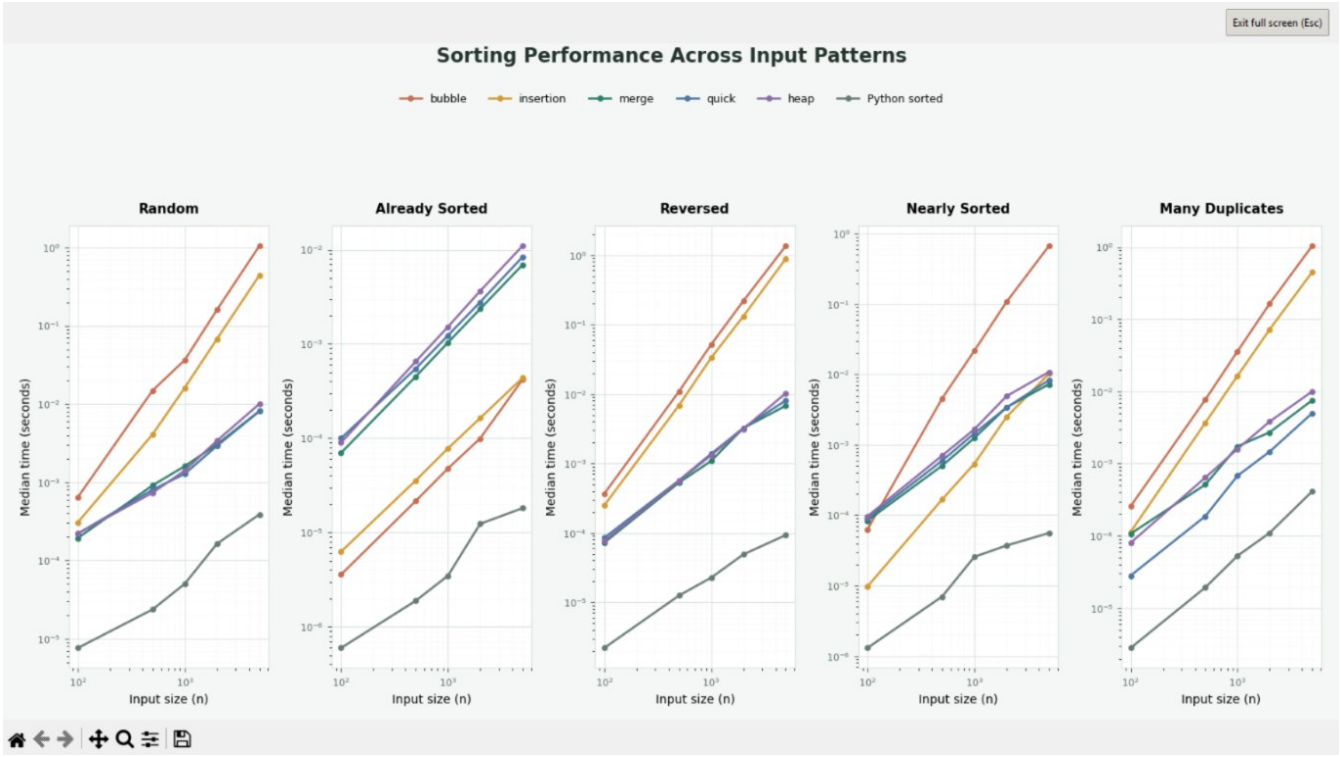


Figure 4.2: Sorting Performance Across Input Patterns

Observed trend: Python sorted() is the fastest measured method in this run. Among the hand-implemented methods, merge sort is strongest on random and reversed inputs, while the iterative three-way quick sort performs particularly well on duplicate-heavy data. Bubble and insertion sort grow much more slowly on general unsorted inputs.

The progress panel reports completed algorithm/input combinations while the benchmark is running. The full generated run contains 150 measurements: five algorithms plus the built-in baseline across five input patterns and five sizes.

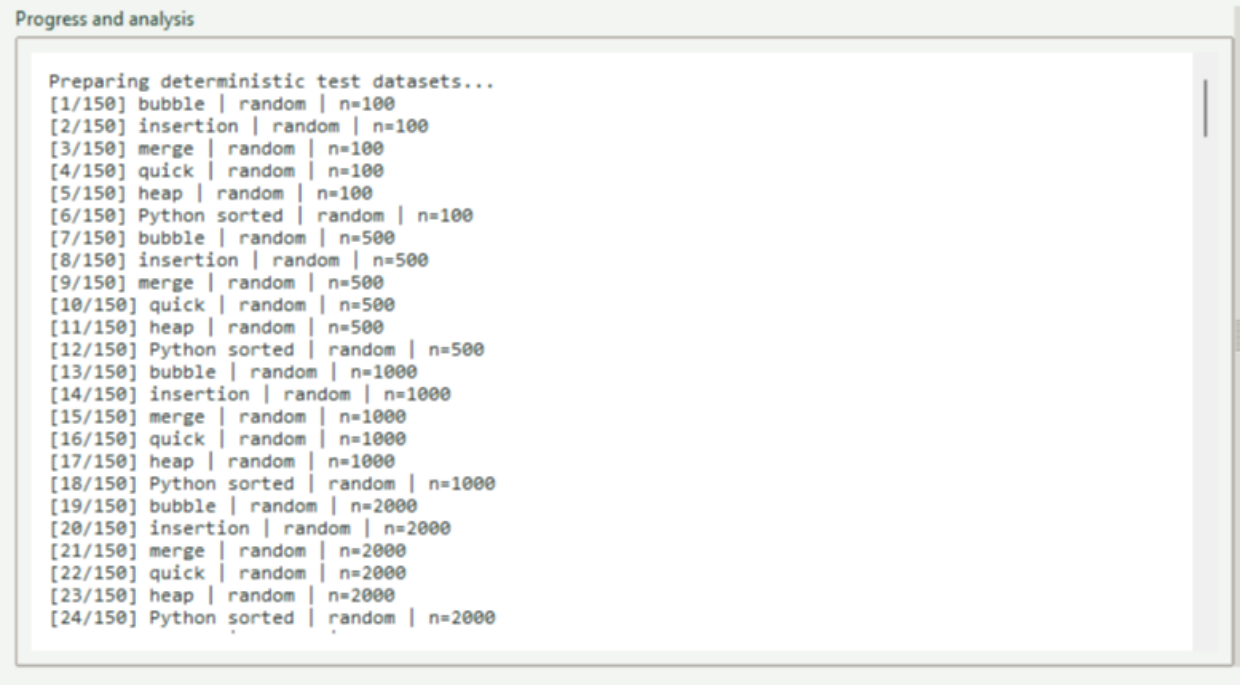


Figure 4.3: Benchmark Progress and Dataset Status

The custom-array comparison below is the recorded sample report for [8, 3, 5, 1, 9]. Timings are a single machine's median-of-five measurement and will vary with hardware and system load.

Figure 4.4: Sample Custom-Array Comparison Output

Rank	Algorithm	Median time (seconds)
1	Python sorted()	0.00000070
2	Insertion sort	0.00000300
3	Bubble sort	0.00000460
4	Heap sort	0.00000740
5	Merge sort	0.00001250
6	Quick sort	0.00003000

5. CONCLUSION

The Sorting Algorithm Race project demonstrates an end-to-end way to implement, validate, time, and visualize sorting methods. Its Tkinter interface coordinates deterministic test-data generation, background benchmark work, correctness checks, chart display, and file export. The benchmark uses five input shapes, five array sizes, and five repetitions per eligible algorithm, producing 150 measurements for the configured test set.

In the recorded n = 5,000 random-input case, merge sort was the fastest hand-implemented method at 0.007726 seconds; Python sorted() measured 0.000391 seconds. For many duplicates, the three-way quick sort was the fastest hand-implemented method at 0.003820 seconds. On already sorted data, bubble sort's early-exit behavior made it the fastest of the project's custom algorithms. These results illustrate that input characteristics matter and that optimized standard-library implementations provide a useful baseline.

Limitations: timings depend on the machine and current system load, and the largest generated input is 5,000 values. Bubble and insertion sort are quadratic on general unsorted data, so user-array comparisons are capped at that size to avoid excessive run times. The benchmark reports elapsed time but does not measure memory usage.

Future scope: the application could add larger-input benchmarking for efficient methods, memory-use measurements, configurable repetitions and seeds, and exportable summary charts or reports.